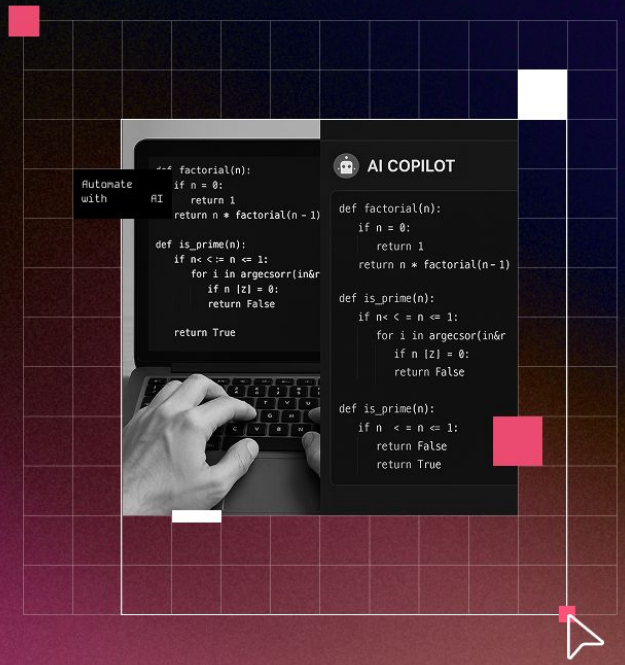
Airtribe

LangGraph for Predictable Agents & State Control

Structured, Controlled, Debuggable AI Systems



Divij Bajaj

Sr. Data & Applied Scientist @Microsoft

Meet your Instructor

Divij Bajaj is a Data & Applied Scientist II at Microsoft with ~6 years of experience in building, productionizing, and scaling large-scale machine learning models for real-world business problems. He has worked extensively on end-to-end ML systems, from problem framing and data modeling to deployment and monitoring in production. Divij has delivered high-impact solutions across enterprise use cases and has been recognized for excellence through multiple competitive and academic achievements.

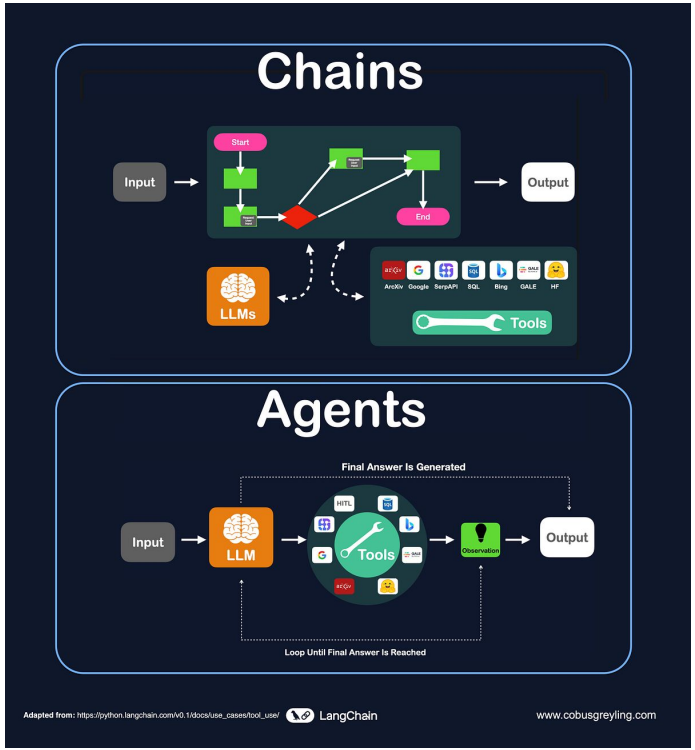
Divij Bajaj

Sr. Data and Applied Scientist @

Airtribe

Quick Recap

- **Chains:** fixed sequence → predictable
- **Tools:** functions/APIs wrapped for LLM
- **Agent loop:** plan → call tool → observe → repeat
- **Pain point:** loops unpredictable, hard to debug or control



What We'll Do Today

- Why Traditional AI Agents Break
- Thinking in Graphs, Not Loops
- State, Control Flow, and Debugging
- Building Real Agent Workflows
- Human-in-the-Loop & Memory Patterns

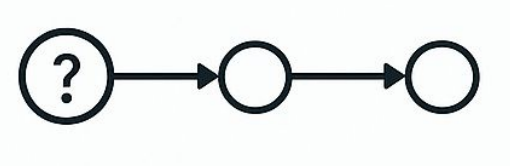
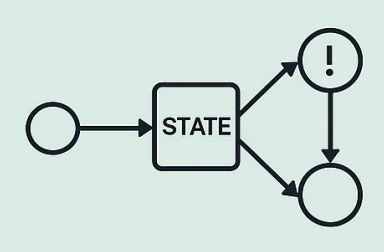
From Agents to Graphs — Why LangGraph Exists (Learning Outcomes)

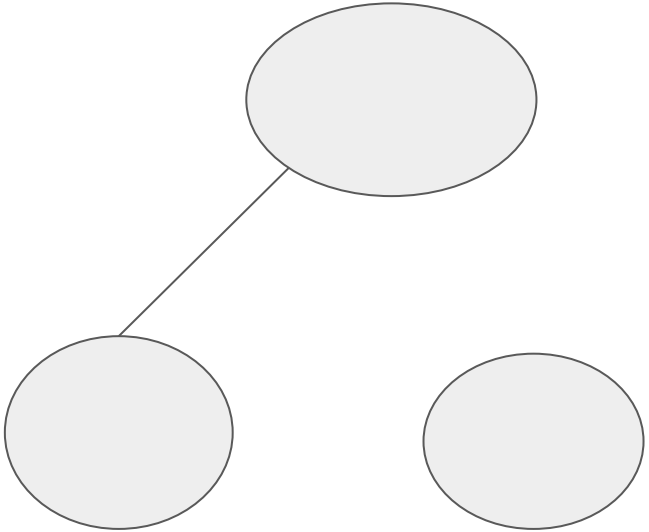
Learners will learn:

1. The limitations of traditional agent loops and why predictability and control are hard to achieve.
2. How to describe LangGraph’s core mental model and how it differs from chains and agents.
3. How to reason about agent behavior as a graph, not an implicit loop.

Why LangGraph?

Airtribe

LangChain default agents	**LangGraph**
	
Hard to predict	Explicit nodes & edges
Limited state control	Clear shared state object
Tough to debug	Branching, loops, retries
Only linear chains	Error handling & recovery



LangGraph Mental Model

Airtribe

Think in **graphs**, not loops:

Nodes = steps:

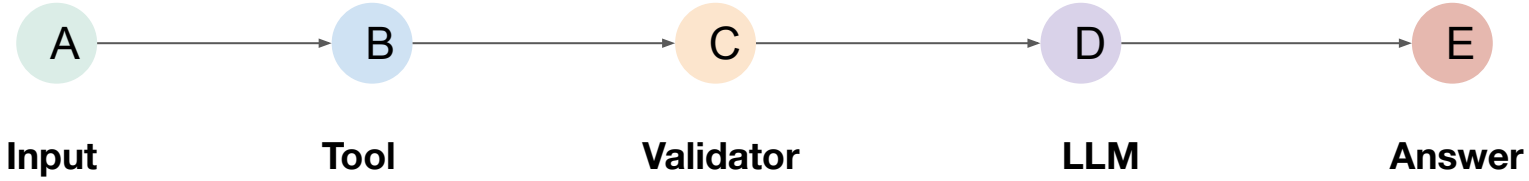
- LLM call
- Tool execution
- Validate output
- Retrieve docs

Edges = next step:

- Success → next
- Failure → fallback
- Retry → loop

State = memory:

- User input
- Intermediate results
- Tool outputs
- Flags



State, Control Flow, and Reliability (Learning Outcomes)

Learners will learn to:

1. Define and design a typed state object to carry data across an agent workflow.

2. Implement conditional logic and loops using edges instead of hidden agent reasoning.

3. Debug agent behavior using state inspection and checkpoints, rather than guessing model behavior.

Core Concepts: Nodes & Edges

Airtribe

Nodes

- Regular Python functions
- Receive state
- Return updates

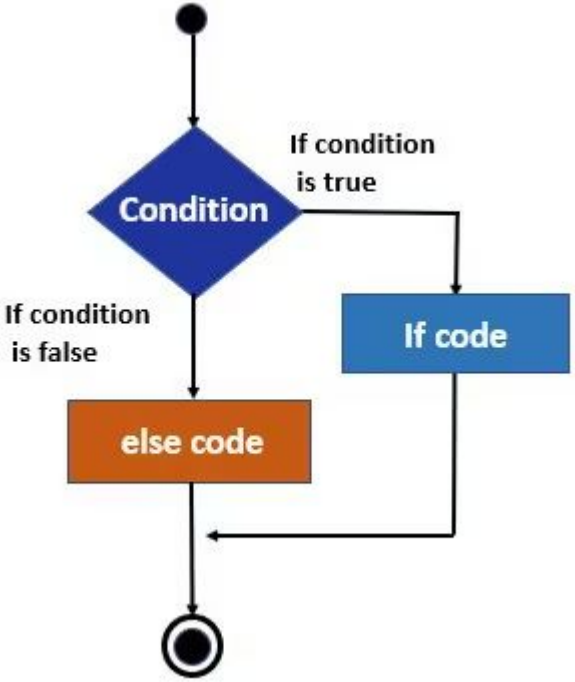
Edges

- Connect nodes
- Choose next step
- Can be conditional

Conditional Edge Example:

```
if retrieval_success → answer_node

else → fallback_node
```



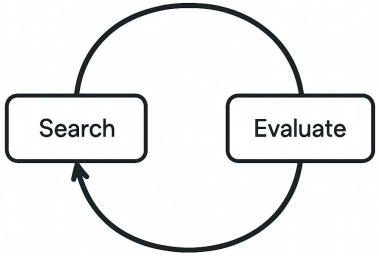
Loops in LangGraph

Cycles allow:

- Retries
- Increment attempts
- Progressive refinement

Example:

```
search → evaluate → (if not enough) → search
```



(If not enough)

Core Concepts: State & Checkpoints

Airtribe

State:

- Typed dict/model
- Carries question, context, answers, attempts

Conditional Logic:

- Edges inspect state
- e.g. `attempts < 3`, `has_context == True`

Checkpoints:

- Save state
- Resume long flows
- Human review points

Question

Context

Attempts

Python Typing Refresher

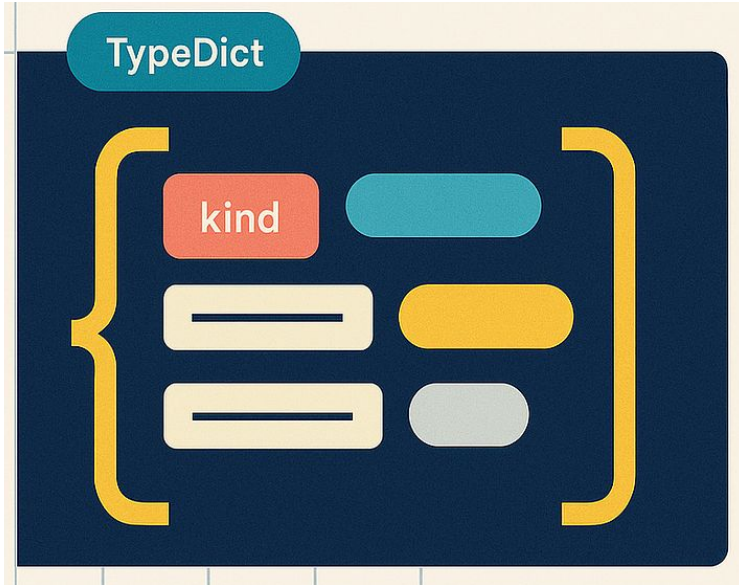
Airtribe

Why types matter:

- Define state shape
- Fewer bugs
- Better IntelliSense

Example:

```
class State(TypedDict):
    question: str
    context: str
    answer: str
    attempts: int
```



Pattern:

- Define state
- Node functions → return partial updates

Minimal Graph: Q&A

Airtribe

State:

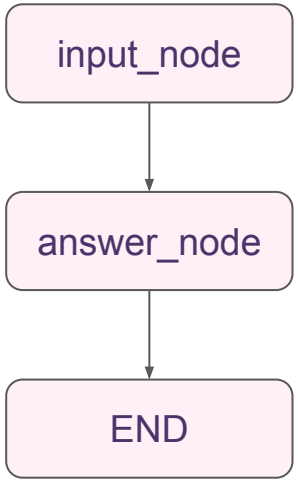
- question
- answer

Nodes:

- input_node
- answer_node

Edges:

input_node → answer_node → END



Visualizing in LangSmith

To debug:

- Connect LangSmith
- Run minimal graph
- Inspect timeline
- Inspect state snapshots

Takeaway:

“You don’t guess. You see exactly what happened.”

Conditional Nodes

Airtribe

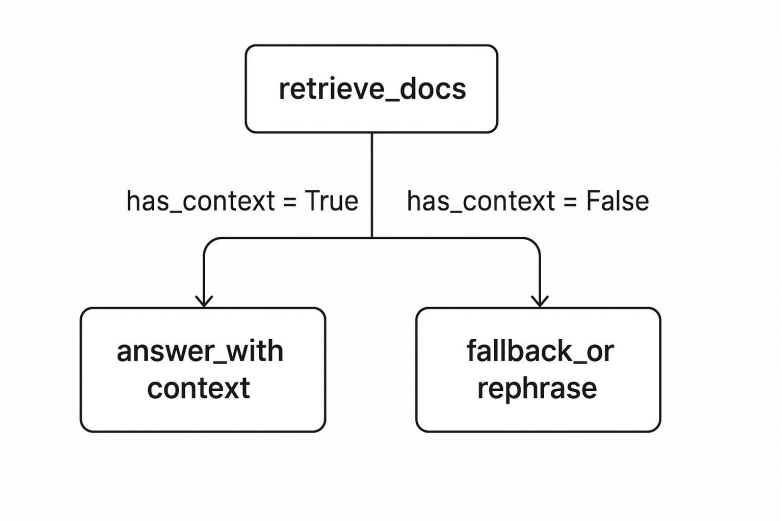
Example:

Node: `retrieve_docs`

→ sets `has_context = True/False`

Conditional Edge:

- if `has_context` → `answer_with_context`
- else → `fallback_or_rephrase`



Building Real Agents - Tools, Loops, and Humans (Learning Outcomes)

Learners will learn to:

1. Compose multi-step agent workflows combining LLM calls, tools, and validation steps.
2. Design retry and fallback strategies using explicit graph cycles.
3. Incorporate human approval and memory into agent workflows for production-grade systems.

Adding LLMs & Tools

Airtribe

Tool Nodes:

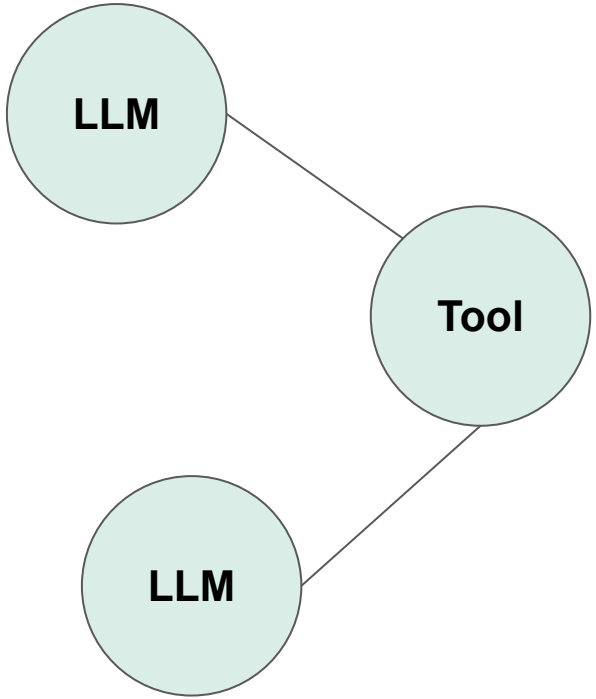
- API calls
- Python functions

LLM Nodes:

- Planning
- Refining
- Summaries

Pattern:

1. `decide_action` — LLM
2. `run_tool` — Tool
3. `summarize` — LLM



Search Agent Example

Airtribe

Nodes:

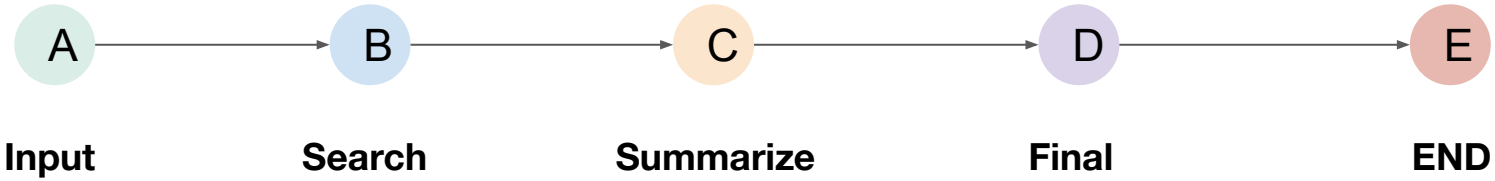
- input_node
- search_node
- summarize_node
- final_node

input → search → summarize → final → END

State:

- question
- search_results
- summary

Flow:



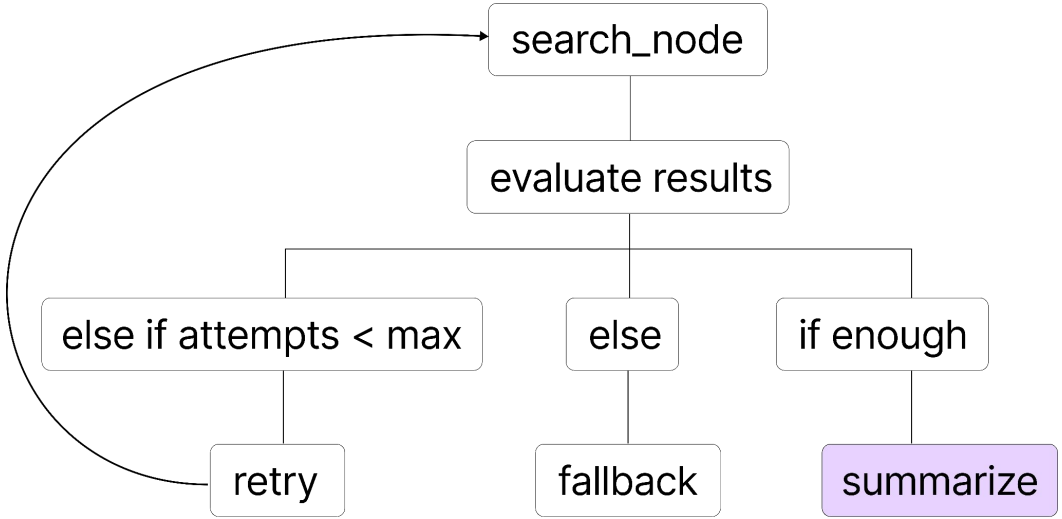
Adding a Loop

State:

- attempts
- enough_results

Flow:

- search_node increments attempts
- evaluate results
- if enough → summarize
- else if attempts < max → retry
- else → fallback



Human-in-the-loop

Airtribe

Use for:

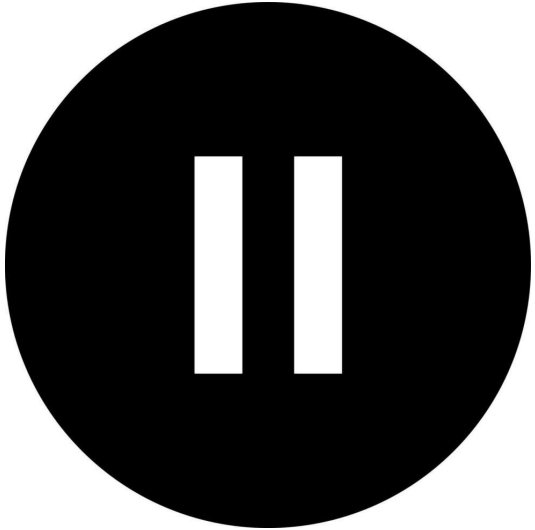
- Approving tool arguments
- Selecting between options

Pattern:

- `propose_action`
- `await_human_review`
- resume graph

Use cases:

- Customer support
- Content review



Memory in Graphs

Airtribe

Short-term:

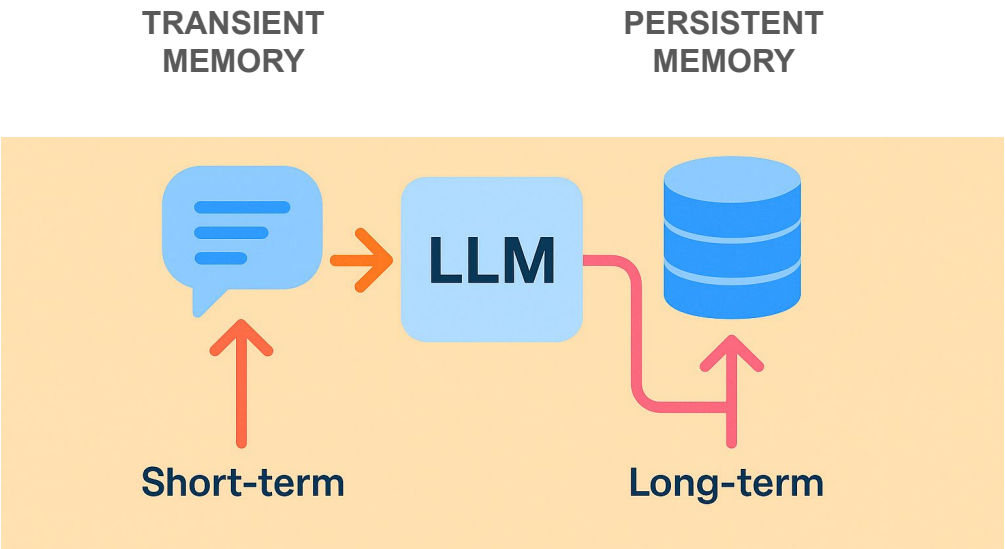
- conversation history
- recent results

Long-term:

- external DB
- vector store

Patterns:

- `load_memory`
- `update_memory`



Wrap-Up

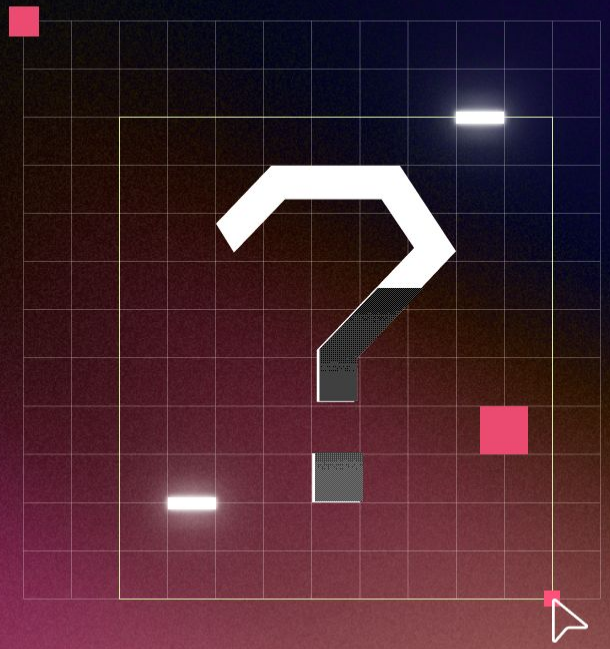
Airtribe

You learned:

- Why LangGraph gives predictable agents
- Nodes, edges, cycles
- State-driven logic
- LLM nodes + tool nodes
- Human approval flows
- Debugging with LangSmith

Airtribe

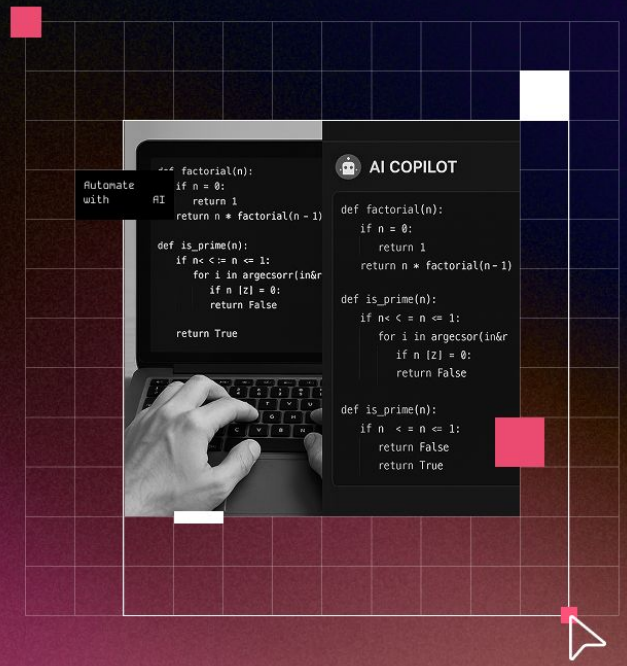
QNA



Airtribe

MCP Fundamentals

A Standard Protocol for Safe, Predictable Tool Use by LLMs



Divij Bajaj

Sr. Data & Applied Scientist @Microsoft

Agenda

- What MCP is
- Why it exists
- How the protocol works
- Build a simple MCP server
- Connect an MCP-aware client
- Trigger tools & inspect flows
- Add your own custom tool

Foundations of MCP (Learning Outcomes)

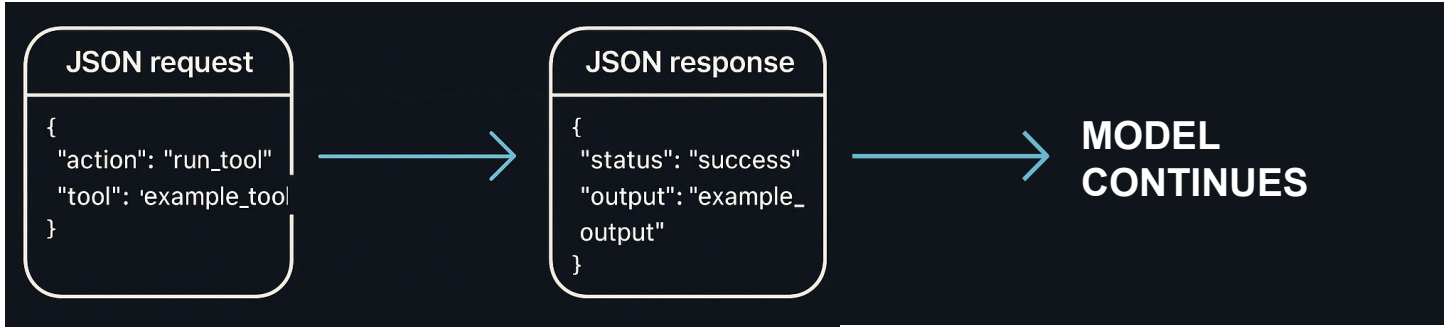
Learners will learn to:

- Define Model Context Protocol (MCP) and its purpose
- Explain why traditional APIs are insufficient for safe AI tool usage
- Compare MCP vs traditional APIs from an AI integration perspective
- Identify use cases where MCP is the right choice

What Is MCP?

Model Context Protocol = a standard for AI → tool communication.

- Standard request/response format
- No custom integration for each tool
- Same pattern across all clients and models
- Safe, permission-controlled execution



What is the Model Context Protocol (MCP)

MCP is an open protocol that standardizes how your **LLM applications** connect to and work with your **tools & data sources**.

REST APIs

Standardize how **web applications** interact with the **backend**

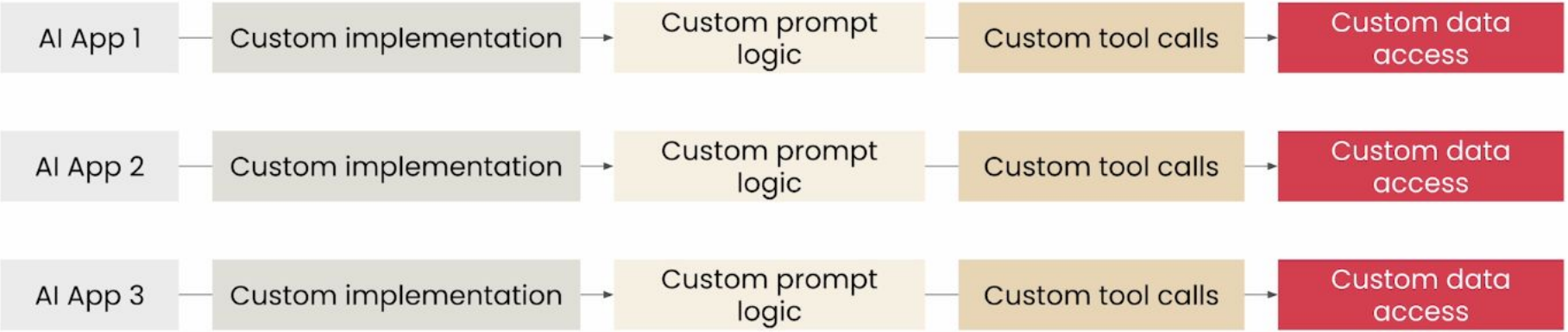
LSP

Standardizes how **IDEs** interact with **language-specific tools**

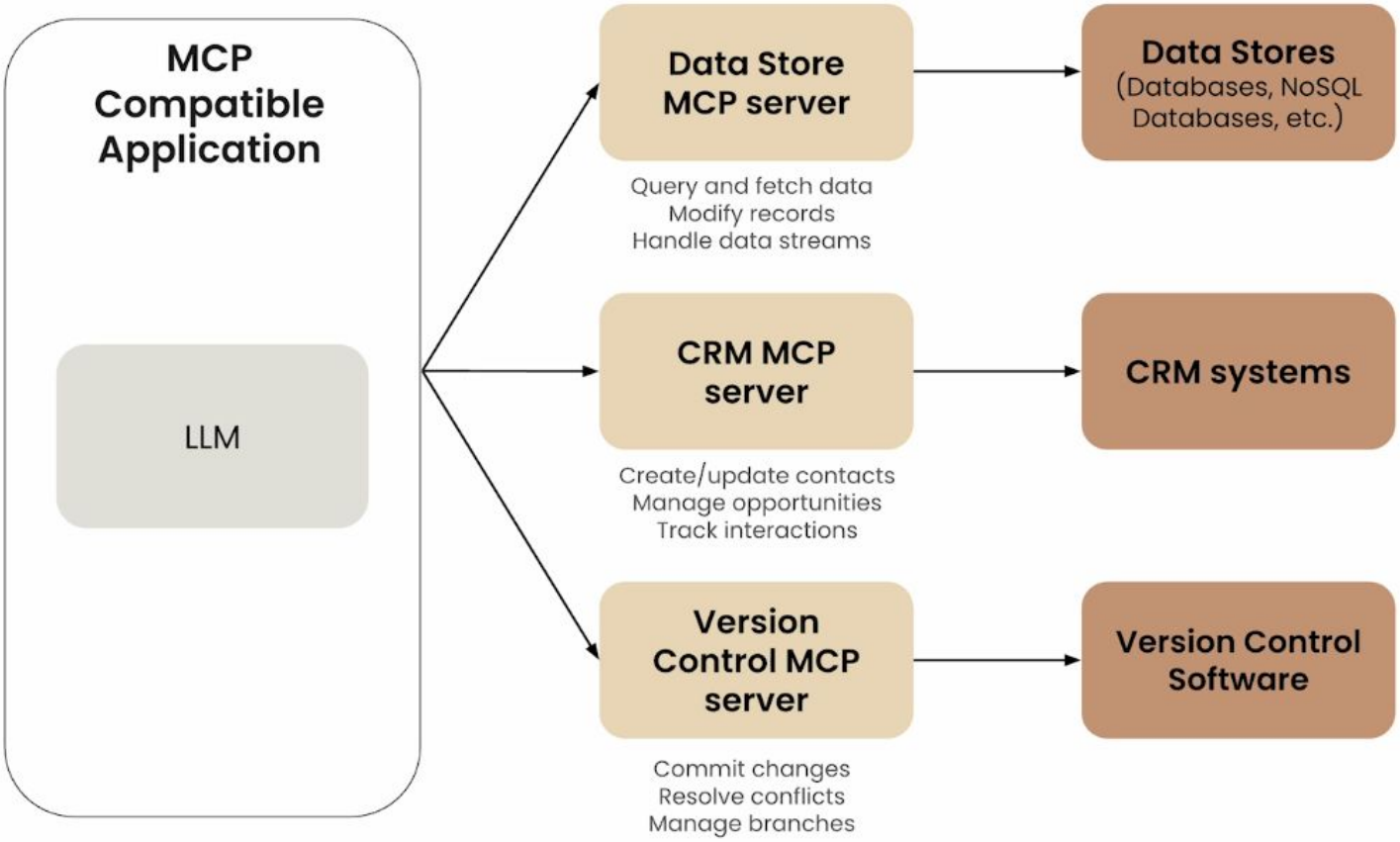
MCP

Standardizes how **AI applications** interact with **external systems**

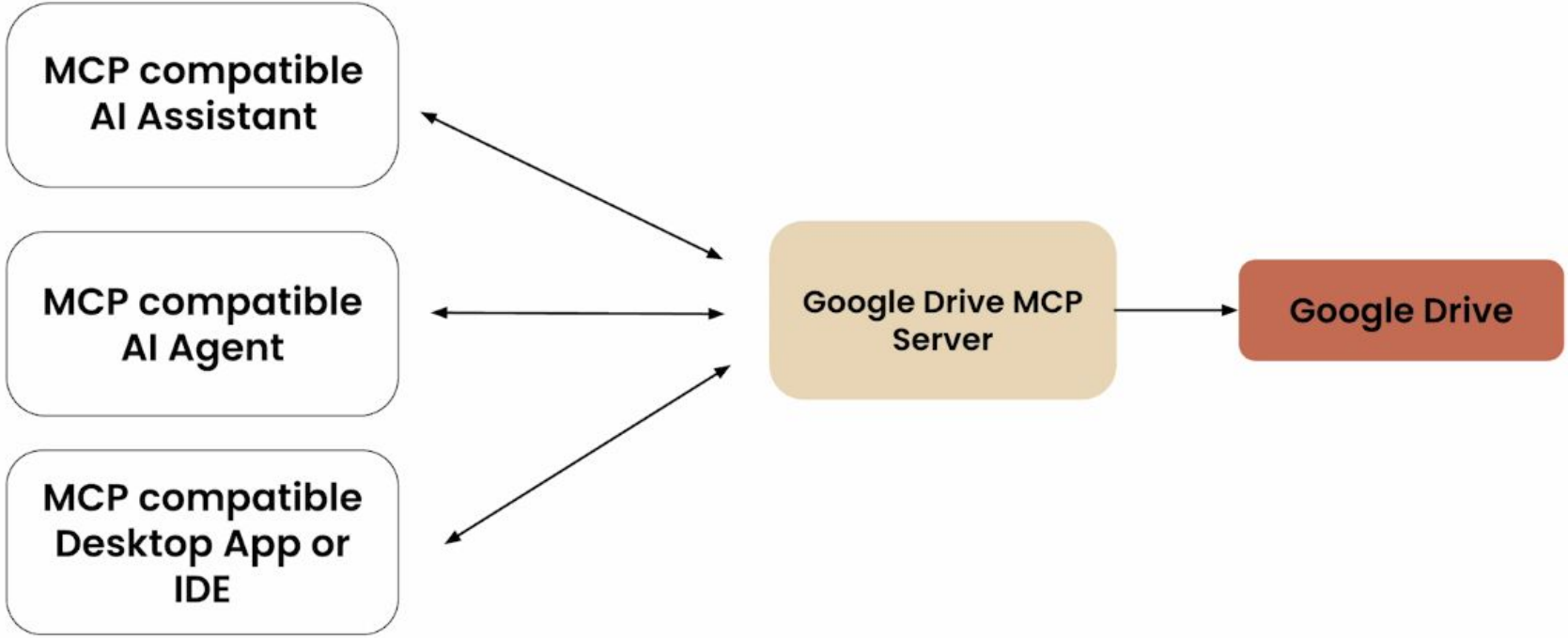
Without MCP: Fragmented AI Development



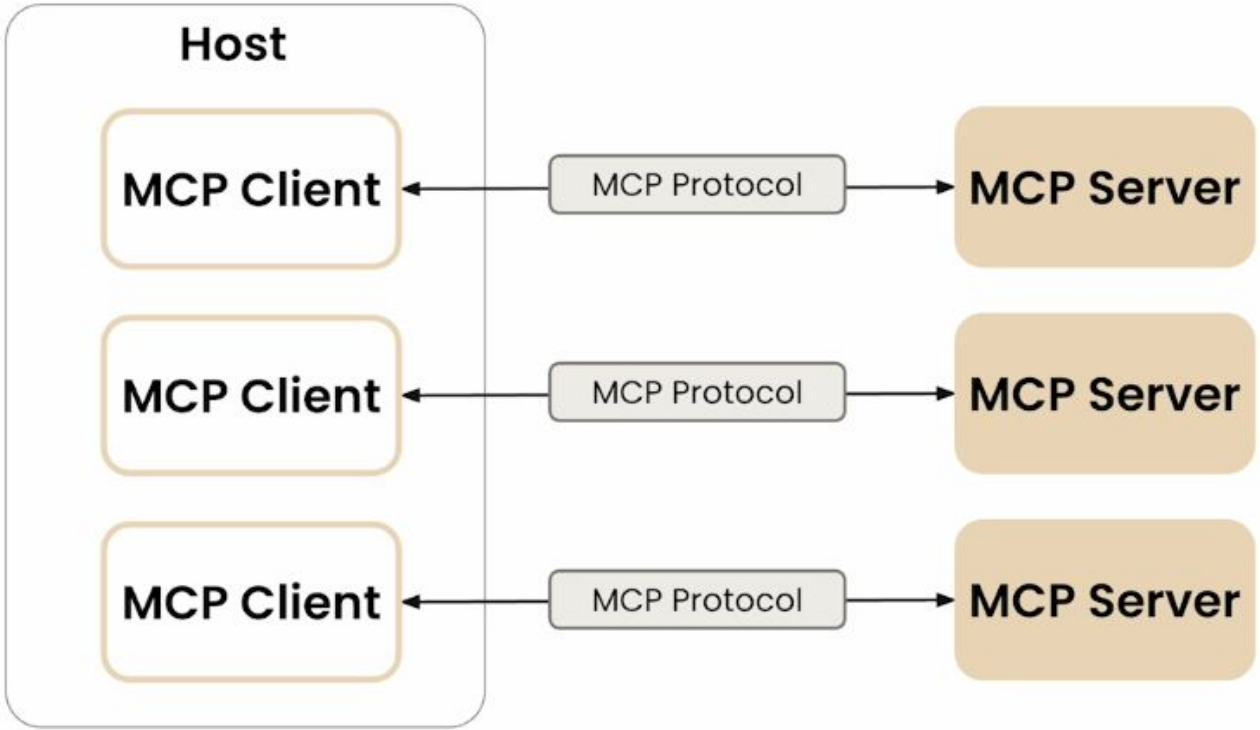
With MCP: Standardized AI Development



With MCP: MCP servers are reusable by various AI applications



Client-Server Architecture



How does it work?

MCP Client

Invokes **Tools**
Queries for **Resources**
Interpolates **Prompts**

MCP Server

Exposes **Tools**
Exposes **Resources**
Exposes **Prompt Templates**

Tools

Functions and tools that can be invoked by the client

- Retrieve / search
- Send a message
- Update DB records

Resources

Read-only data or exposed by the server

- Files
- Database Records
- API Responses

Prompt Templates

Pre-defined templates for AI interactions

- Document Q&A
- Transcript Summary
- Output as JSON

Why MCP Matters

Models today need access to:

- Local tools
- Private data
- System actions
- Editor commands
- File system

APIs alone can’t handle these safely.

The Core Problem MCP Solves

Airtribe

Without MCP:	With MCP:
Every integration is custom	Standard interface
Tools behave differently	Same tool works across all AI apps
No shared standard	Clients handle permission + safety
Hard to allow model access to local resources	Tools become reusable
No permission workflow	

MCP vs Traditional APIs

Airtribe

APIs

- Designed for web services
- Need network calls
- Built for humans writing code
- No built-in safety prompts
- Hard for LLMs to reason with

MCP

- Designed *for models*
- Local or internal tools
- Permission-based
- Predictable JSON format
- Standard flow across clients

MCP Architecture & Workflow (Learning Outcomes)

Learners will be able to:

- Identify the core components of MCP, including the client, server, tools, resources, and model
- Describe the end-to-end MCP workflow from user query to tool execution
- Interpret the structure and return format of MCP tool calls
- Distinguish between tools and resources in MCP and determine appropriate use cases for each

When To Use MCP

Airtribe

Use MCP when:

- Multiple AI apps need the same tools
- Tools are **local** (file ops, search, logs)
- Tools are **internal** (private DBs, internal APIs)
- You want consistent safety prompts
- You want to change the model without rewriting integrations

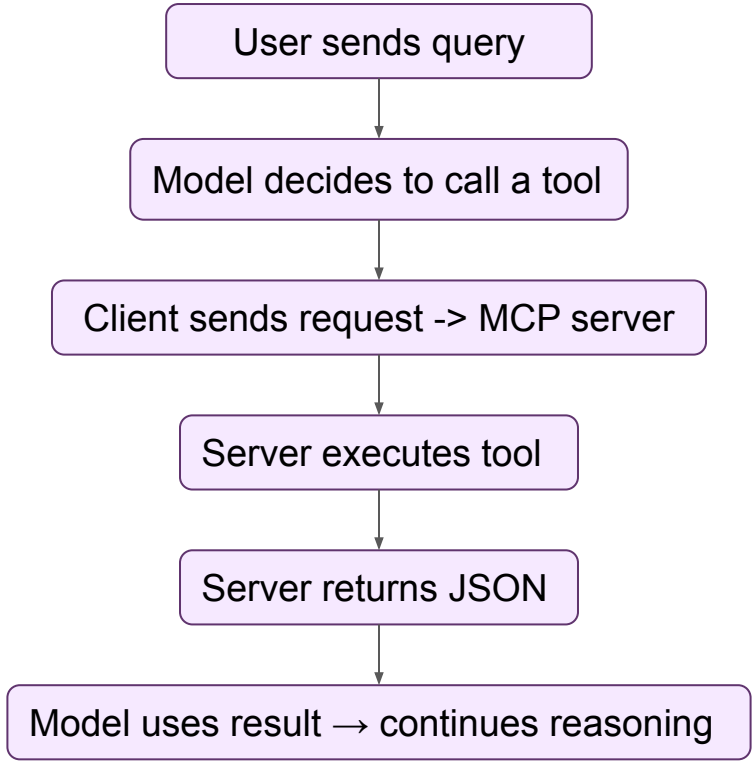
MCP Building Blocks

- **MCP Client:** e.g., Claude Desktop
- **MCP Server:** your tool host
- **Tools:** Functions with schema + name
- **Resources:** Streams / lists of data
- **Messages:** JSON request/response
- **Model:** Decides when to call tools

CLIENT ↔ **SERVER** ↔ **TOOLS/RESOURCES**

The MCP Flow

Airtribe



Demo: Claude Desktop as MCP Client

Airtribe

- Claude Desktop automatically detects MCP servers
- Shows available tools
- Lets users approve permissions
- Provides visual logs

Breakdown of a Tool Call

Airtribe

Tool Call Structure:

- **tool_name:** e.g., `"list_files"`
- **arguments:**

```
{"path": "/documents", "filter": ".txt"}
```

- **result:** JSON containing output
- **error:** structured error format

Building & Applying MCP in Practice (Learning Outcomes)

Learners will learn to:

- Define Model Context Protocol (MCP) and its purpose
- Explain why traditional APIs are insufficient for safe AI tool usage
- Compare MCP vs traditional APIs from an AI integration perspective
- Identify use cases where MCP is the right choice

Building Your First MCP Tool

Steps:

1. Define tool name
2. Define input args
3. Return JSON output
4. Register tool in server
5. Restart server
6. Test in client

```
my_dictionary = {  # Open with curly brace
  "Apple": 16,
  "Cherry": 41,  # Map item using colon
  "Grape": 66,
  "Melon": 2,  # Separate pairs with comma
  "Lemon": 6
}  # Close with curly brace
```

Demo: Build an MCP Server

Airtribe

- Python server template
- Add one tool
- Start server
- Connect via Claude Desktop

How Tools Should Return Data

Airtribe

Guidelines:

- Always return JSON
- Consistent field names
- Include metadata if needed
- Use clear error messages
- No silent failures

Resources in MCP

Airtribe

Resources = data that changes over time

Examples:

- Folder watchers
- Log streams
- Database updates
- Realtime metrics

Features:

- Client can subscribe
- Server pushes updates

What MCP Gives on Top of APIs

Airtribe

- Unified connector for all clients
- Shared tool layer for multiple apps
- Consistent permission prompts
- Long-term maintainability
- Easy model swap
- Local tool support

How MCP Fits Into a Real Stack

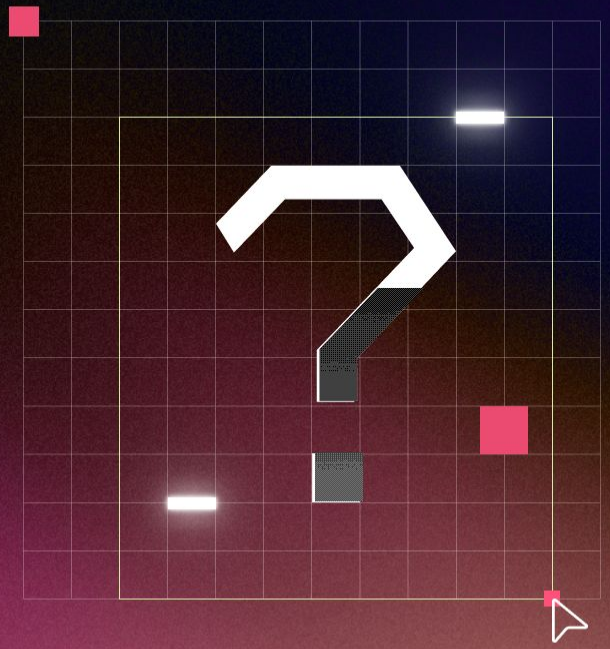
```
Your Product
     ↓
MCP Server (tools: DB, file ops, APIs…)
     ↓
MCP Client (Claude Desktop / IDE plugin / custom app)
     ↓
Model (LLM)
```

Wrap-Up

Airtribe

Today you learned:

- What MCP is
- Why it solves tool-integration problems
- Client ↔ Server ↔ Tool flow
- How to build a basic server
- How to trigger tools from a model
- How to extend MCP with custom tools

Airtribe

QNA